



Fixing the Token Authentication

Revocation, Scoping, and Securing
the Execution Boundary

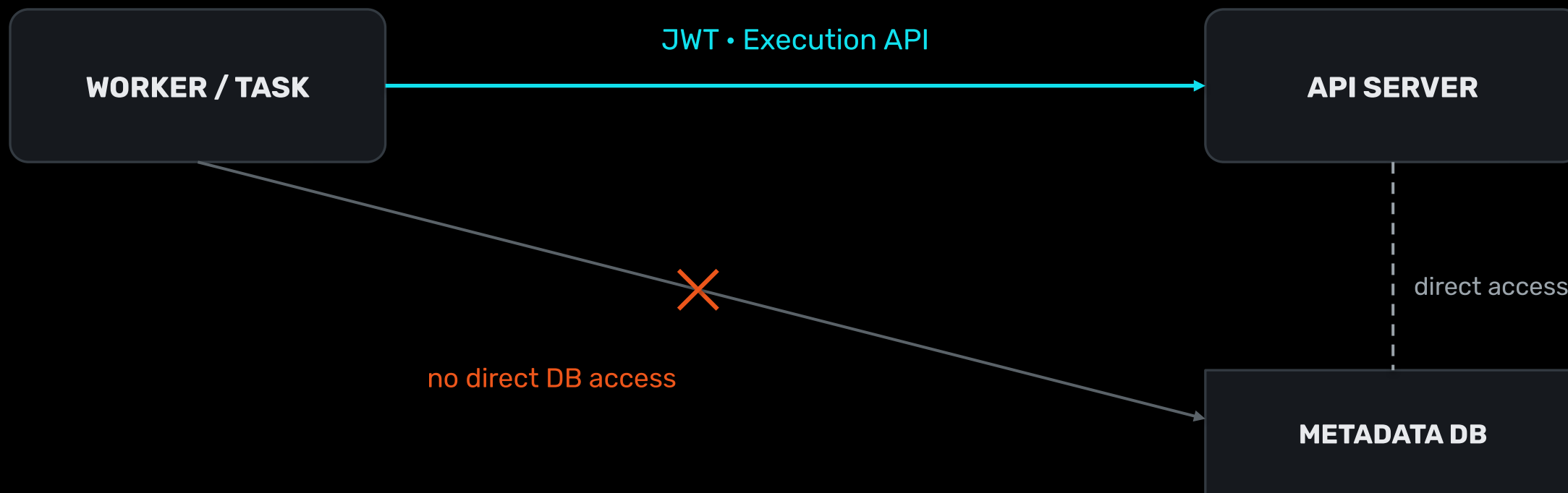


Anish Giri · Apache Airflow Contributor · @anishgiranish

Airflow Summit 2026 · Austin

The views in this talk are my own and do not represent any organization.

What Airflow 3 changed



Workers no longer touch the database. Every call is a token.

Scope of this talk



Worker ↔ Execution API

Dag File Processor / Triggerer

UI / REST tokens

Today: the two highlighted boundaries. Token lifecycle, not the RBAC model.

Sharing what I learned working on these two boundaries this past year.

Agenda



- 1 · Ending tokens: revocation on logout
- 2 · Scoping tokens: two tokens for two jobs
- 3 · Reusing the pattern: connection testing



1

Revocation

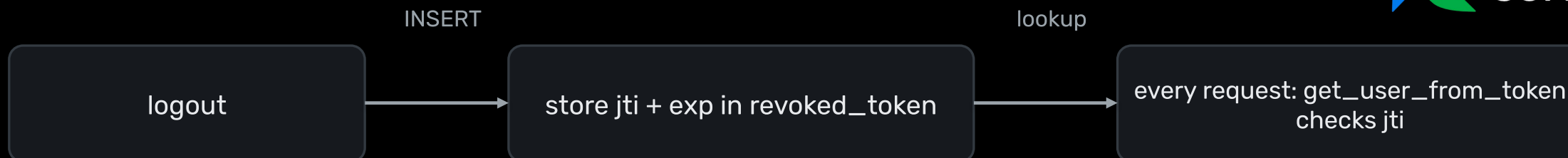
PR #61339 • merged • Airflow 3.2

Log out. Token still works.

User API/UI JWTs stayed valid until expiry.

Execution API worker tokens are a different class:
short-lived, auto-refreshed, deliberately non-revocable
(see security model docs).

The design



```
async def get_user_from_token(self, token: str) → BaseUser:
    """Verify the JWT token is valid and create a user object from it."""
    payload = await self._get_token_validator().avalidated_claims(token)

    if (jti := payload.get("jti")) and RevokedToken.is_revoked(jti):
        raise InvalidTokenError("Token has been revoked")

    return self.deserialize_user(payload)
```

base_auth_manager.py: the jti check sits in the one path every authenticated route passes through. (PR #61339)

The lifecycle



Cleanup via `db_cleanup` + periodic in-process sweep, suggested by Jarek Potiuk on the PR.

What it opens



Merged Feb 2026

*opens the door for ... a token invalidation endpoint for administrators
in case of a token leak*

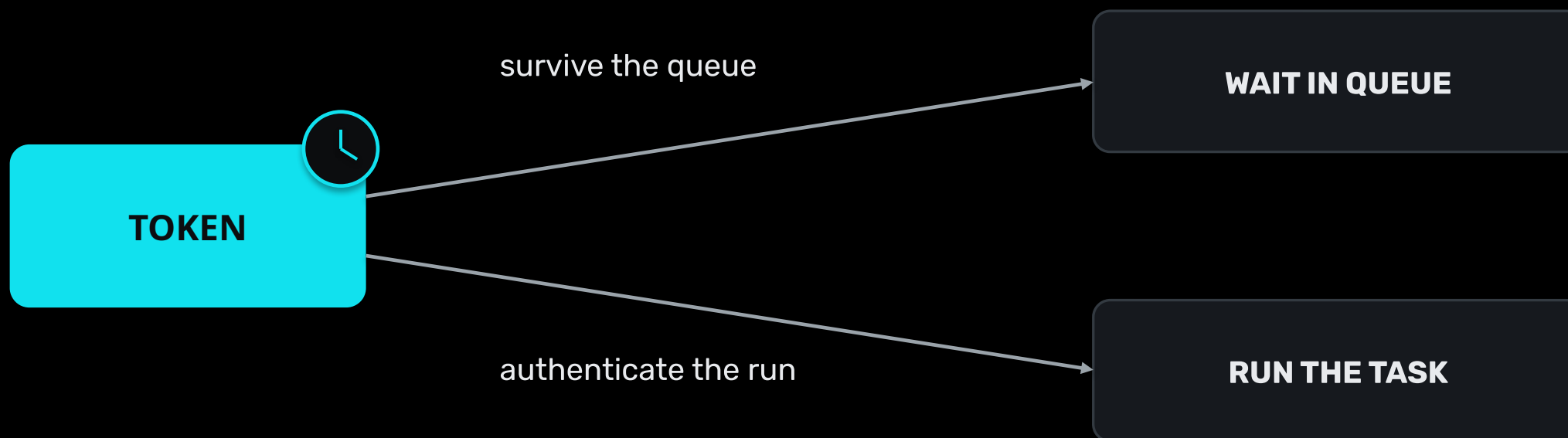
Buğra Öztürk, PR comment

2

Scope separation

PR #60108 • merged • Airflow 3.2.2

The problem



One token, two jobs, one lifetime. Queue backs up → token expires → first /run returns 403 → task never starts. (#53713)

It looked like flakiness

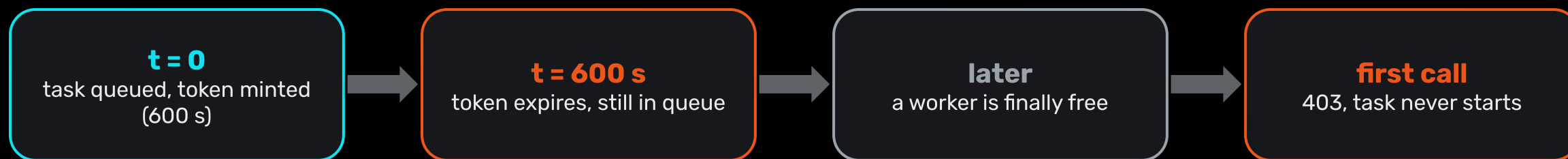


The user raised `task_queued_timeout` to 100,000 seconds, more than a day. Tasks still failed at about 15 minutes.

```
1974 # Variable: AIRFLOW__SCHEDULER__TASK_QUEUED_TIMEOUT
1975 #
1976 # Close to 26 hours
1977 task_queued_timeout = 100000
1978
1979 # How often to check for tasks that have been in the queued state for
1980 # longer than `[scheduler] task_queued_timeout`.
1981 #
1982 # Variable: AIRFLOW__SCHEDULER__TASK_QUEUED_TIMEOUT_CHECK_INTERVAL
1983 #
1984 task_queued_timeout_check_interval = 120.0
```

Reporter's own config, from issue #59553

Anatomy of the failure



The clock starts at dispatch, not at pickup. Any queue longer than the token loses. (#53713)

Why the obvious fix is wrong

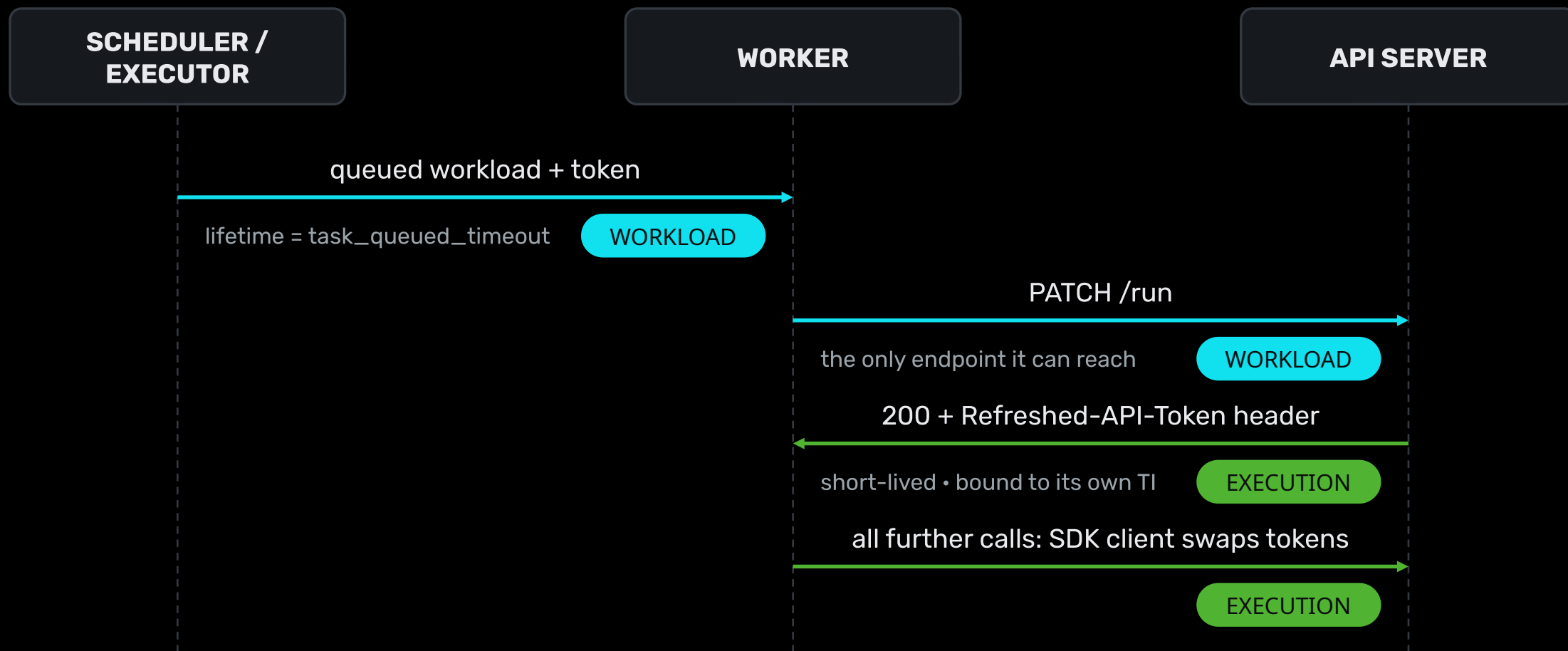


My first approach:
refresh expired tokens
transparently.

*It lets any Execution API token be
resurrected ... it amounts to having
tokens not expire.*

Ash Berlin-Taylor, first review

The design: two tokens



JWTReissueMiddleware auto-refreshes execution tokens and skips workload tokens.

An intercepted workload token can only reach /run.

/run refuses any TI not in QUEUED or RESTARTING. The state machine enforces it.

Execution token: short-lived, bound to its own TI (ti:self).



Enforcement mechanics



```
async def require_auth(
    security_scopes: SecurityScopes, request: Request,
    token: TIToken = Depends(_jwt_bearer),
) → TIToken:
    route = request.scope.get("route")
    allowed = getattr(route, "allowed_token_types",
                      frozenset({"execution"}))
    if token.claims.scope not in allowed:
        raise HTTPException(status.HTTP_403_FORBIDDEN, ...)
    if "ti:self" in security_scopes.scopes:
        if str(token.id) ≠ str(request.path_params["task_instance_id"]):
            raise HTTPException(status.HTTP_403_FORBIDDEN, ...)
```

execution_api/security.py: routes opt in to workload tokens; everything else refuses them. Built on Ash Berlin-Taylor's SecurityScopes foundation. (PR #60108)

From rejection to merge: 109 days



Jan 4: PR opened with the transparent-refresh approach. Jan 7: rejected in review.

Redesigned into two tokens. Three months of review rounds with five reviewers.

Apr 23: merged with four approvals.

#60108 • opened Jan 2026 • merged Apr 2026 • backported to 3.2.2

A model, not a patch



Refresh logic reasons only about execution tokens.

New workload types reuse the pattern: connection tests, callbacks.

Merged Apr 2026



3

Connection testing

PR #62343 • merged Jun 2026 • Airflow 3.3

A button that has been off for three years.

Test Connection has defaulted to Disabled since Airflow 2.7 — for a good reason.



Why the API server is the wrong place



Testing a connection runs the hook's driver code – ODBC, JDBC, whatever the provider needs.

That code ran inside the API server: the component holding the signing key and the database.

Arbitrary provider code, on the most privileged component you run.

And the API server often cannot reach the system you are testing anyway. Workers can.

The design: make it a workload

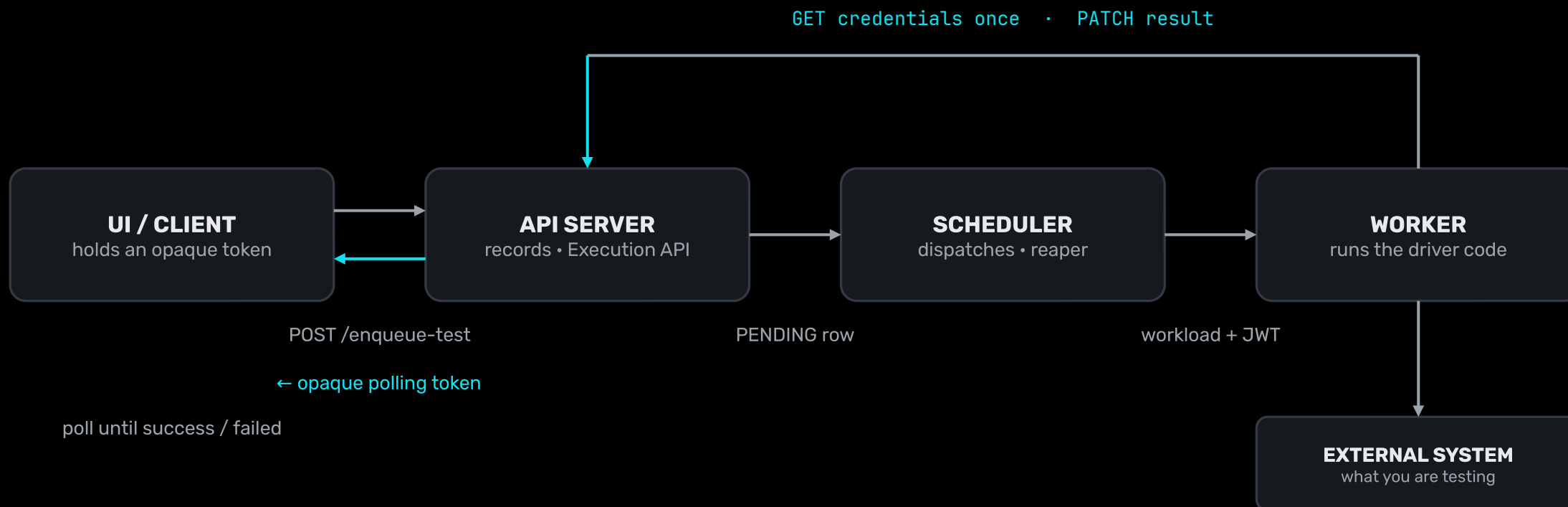


You POST to enqueue a test and get back an opaque polling token.
Nothing has run yet.

The scheduler dispatches it to a worker, which runs the driver code and reports back over the Execution API.

You poll with that token until it is success or failed. 60s timeout • 4 at a time • a reaper fails stale tests.

Where the test actually runs



The API server records and reports. The worker runs the driver code – and it is the one that can reach your system.

Two credentials, and they never meet: the client holds an opaque token, the worker holds the JWT.

Enforcement: same shape, new subject



That router accepts workload tokens only – it does not list the execution scope at all.

`ct:self` binds the token to its own connection test, exactly as `ti:self` binds a task.

Credentials can be read once: the read takes a row lock and flips the test to `RUNNING`. A second read gets 409.

So a stolen request token can start its own test, once, and nothing else.

What's shipped, what isn't

Shipped

- Execution API side: workload scope, ct:self, single-use credential fetch
- LocalExecutor dispatches connection tests

Not yet

- Other executors have not opted in (supports_connection_test defaults to False)
- The async API is public; the UI still calls the old synchronous endpoint
- [core] test_connection still defaults to Disabled

Where this leaves us



A token can end. A token does only its job. And dangerous work moved off the API server.

`github.com/apache/airflow/pull/61339 · /pull/60108 · /pull/62343`
`@anishgirianish`

Questions?



github.com/apache/airflow/pull/61339 • [/pull/60108](https://github.com/apache/airflow/pull/60108) • [/pull/62343](https://github.com/apache/airflow/pull/62343)

@anishgiranish